Vector Databases

Modulo 2 · Lezione 9



Dalla ricerca manuale alla ricerca indicizzata

Nella lezione scorsa la ricerca l'avete scritta **a mano** — e l'avete misurata: funziona, ma il tempo cresce *dritto* col numero di vettori, e ogni query rifà tutto il lavoro da capo.

`Modulo 2 · Lezione 8 — la lezione scorsa`

Il **TUO ciclo**: cosine con tutti i vettori, a ogni query — e l'npz da ricaricare a ogni avvio.

Modulo 2 · Lezione 9 — oggi

Un **DATABASE** tiene i vettori indicizzati su disco: tu *interroghi* e basta. Veloce a scala, persistente, filtrabile.

La ricerca resta la **STESSA** — stessi risultati, e lo **verificherete** fianco a fianco col vostro brute-force.



Agenda

Quando c'è da lavorare, una **slide di switch** vi manda al posto giusto — finita l'attività, si torna qui.
🎤 = il docente scrive, voi replicate (o guardate) · ⌨️ = scrivete voi (docente in affiancamento)

	Blocco	Dove si lavora	Tempo
📊	Brief: da brute-force a indice, il modello dati — **nei primi minuti parte il setup**	slide + terminale	35′
🎤	**Quick start: la collection su disco**	`working/vectorstore.py`	15′
⌨️	**Lab 1: il corpus nel DB + search**	`working/vectorstore.py`	75′
☕	Pausa	—	15′
⌨️	**Lab 2: filtri e CRUD**	`working/vectorstore.py`	60′
🎤	**Demo pgvector** (si guarda)	terminale del docente	15′
📊⌨️	**Criteri di scelta + ultimo blocco** (Q&A o esercizio extra)	slide · `working/vectorstore.py`	25′

Setup nei primi minuti, da `scripts/CONSEGNA.md` : il `pip install` stamattina è **lungo** (chromadb) — lanciatelo e lasciatelo andare mentre il brief continua.

La chiave è **quella di Modulo 2 · Lezione 1** (persa? **alzate la mano**) · il corpus è quello della lezione scorsa — il **vostro**, o la riserva già in `dataset/` · costo dell'intera giornata: **~$0.000005** di query.

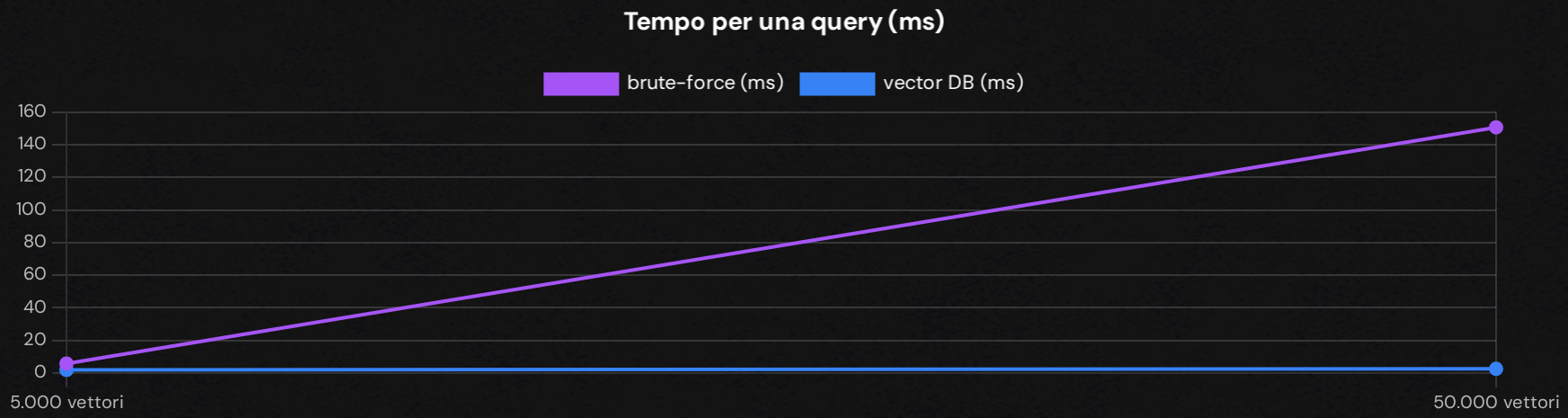


1. Da brute-force a indice



La stessa query, due motori

Vettori come i nostri (1536 dimensioni), **una** query. Il brute-force della lezione scorsa contro un vector DB (ChromaDB):



> Il tempo dell'indice **non cresce con n**. A 10 milioni di vettori il brute-force supererebbe i **40 s** a query; l'indice resta sui **millisecondi**.

Il costo dell'indice: scrittura vs query

Il rovescio, subito: quella velocità **si paga** — una volta sola.

L'indice si paga alla SCRITTURA

Indicizzare 5.000 vettori: **1.3 s**. Indicizzare 50.000: **37.5 s**. È lavoro vero: il DB *organizza i vettori per vicinanza*, e lo fa quando scrivi.

…e ogni query lo ripaga

Si paga **una volta**; da lì in poi ogni ricerca va sui millisecondi.

Lo conoscete già dai database relazionali

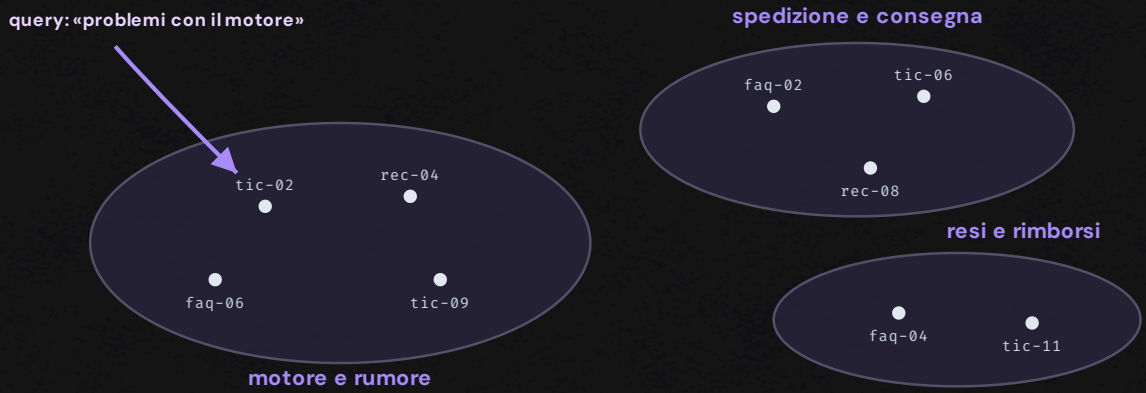
`CREATE INDEX` costa, le `SELECT` dopo volano. Qui l'indice è sui **vettori** invece che su una colonna.

> Per questo i vector DB rendono sui corpora che si **leggono molto e si scrivono ogni tanto** — il caso tipico del retrieval.



ANN: vicini approssimati

Come fa a non confrontare tutto? Tiene i vettori **organizzati per vicinanza** e naviga verso il quartiere giusto, invece di scandire tutto il corpus.



I quartieri sono per **significato**, non per tipo: dentro convivono ticket, recensioni e FAQ — i vicini veri della query sono `tic-02` 0.588, `rec-04` 0.574, `faq-06` 0.557. Per cercare SOLO tra le FAQ serve il filtro sui metadata (Lab 2).

ANN — Approximate Nearest Neighbors

Approssimato: può mancare il vicino perfetto, in cambio di una velocità che non dipende da n.

Sui nostri 48: approssimato = esatto

Lo verificherete: **stessi risultati** del brute-force. Lo scarto si vede — e si accetta — a **milioni** di vettori.

Cosa aggiunge un vector DB

Quattro cose che il vostro ciclo della lezione scorsa non aveva:

⚡

Indice ANN

La ricerca che **non scala più con n**: millisecondi a 50k come a 5k.

💾

Persistenza

I vettori vivono **su disco**: niente npz da ricaricare e ricongiungere a ogni avvio.

🔍

Filtri sui metadata

«Cerca solo tra le **FAQ**» — il ciclo della lezione scorsa non sapeva farlo.

✏️

CRUD

Aggiungi, aggiorna, cancella **senza rigenerare tutto**.

La ricerca della lezione scorsa, fatta da un **servizio**.



2. Il modello dati



La collection

Il contenitore si chiama **collection** (come una tabella). Dentro: record a **quattro campi** — eccone uno del nostro corpus:

```
id          "faq-02"
document    "Quanto costa la spedizione? La spedizione è gratuita…"
embedding   [0.021, -0.043, 0.017, … ] ← 1536 numeri (il vostro npz)
metadata    {"tipo": "faq"}
```

Tutto insieme

A Modulo 2 · Lezione 7 e Lezione 8 erano **tre file da tenere allineati a mano** (jsonl + npz + meta). Il record li tiene uniti lui.

I metadata sono un dict piatto

Oggi `tipo`; in produzione: source, data, lingua, **permessi**. È quello su cui si filtra.

> L'`id` è la chiave: è quello che rende possibile **aggiornare** e **cancellare** — lo farete nel Lab 2.



Le operazioni: quattro verbi

Gli stessi verbi di qualunque database — la novità è solo che la query cerca per **vicinanza**, non per uguaglianza.

Verbo	Cosa fa	Da sapere
`upsert`	**insert-o-sostituisci**: stesso id → si riscrive	mai doppioni
`query`	i più vicini a un embedding	`n_results` (+ `where` per filtrare)
`get`	lookup per id o filtro, **senza** ricerca	utile per ispezionare
`delete`	via, per id	e sparisce dai risultati

Upsert: rilanci sicuri

Rilanciate il caricamento due volte → il count **NON raddoppia**. Per questo gli script di oggi si possono rilanciare senza pensarci.



Gli embedding si passano espliciti

Se passate testi *nudi*, ChromaDB li embedda lui — col **suo** modello di default. È quello che non vogliamo.

L'embedder di default NON è il vostro

Il default di Chroma è **MiniLM (384 dim)**; il vostro corpus è `text-embedding-3-small` (**1536**). Spazi diversi = la regola della lezione scorsa **infranta lato database**.

La regola operativa di oggi

Upsert e query viaggiano **SEMPRE** con embedding calcolati da noi: il corpus dall'**npz di Modulo 2 · Lezione 7**, le query con `embed()` di aikit — backend dal `corpus_meta.json`.

> Il mismatch 384/1536 almeno **esplode con un errore**. Con dimensioni uguali ma modelli diversi **non esplode**: dà risultati senza senso, in silenzio. Il meta è il vostro certificato.



Il DB parla in distanze, noi in score

`collection.query` non restituisce gli score della lezione scorsa: restituisce `distances` — la **distanza** cosine, dove 0 = identico.

```
score = 1 - distanza   (0.244 → 0.756)
```

Le fasce della lezione scorsa valgono ancora

Fatta la conversione: query facile **0.756**, fuori dominio **0.196**, la vostra soglia **~0.30** resta buona.

Nel Lab 1 la vostra `search()` farà questa conversione: i risultati devono avere **la stessa faccia della lezione scorsa** — id, tipo, testo, **score**.



Le tre famiglie di vector DB

Il mercato è affollato, le famiglie sono tre. I **concetti** (collection, embedding, metadata, ANN) sono gli stessi ovunque: si imparano **una volta**.

← gestisci tu

gestito →

ChromaDB · `embedded`

Vive **nel processo Python**: `pip install` e via, i dati in una cartella. **Oggi si lavora qui.**

pgvector · `nel tuo Postgres`

Un'**estensione** del Postgres che hai già: i vettori accanto ai tuoi dati. **Demo a fine mattinata.**

Pinecone · `managed`

Niente da installare: API remota, **si paga a consumo**. In slide, dopo la demo.

Nel mezzo, citati: i **server dedicati self-hosted** (Weaviate, Qdrant, Milvus) — carico serio, dati tuoi.

> I **criteri di scelta** arrivano a fine mattinata — **dopo** che le avrete toccate, non a catalogo.



🎤 Quick start — la collection

🎤 **LIVE CODING — si guarda e si replica**

`working/vectorstore.py` · 15 minuti

poi il file resta a voi

Il corpus: il VOSTRO, se ce l'avete

Il setup è già partito nel brief (`scripts/CONSEGNA.md`). Chi ha il proprio corpus della lezione scorsa lo copia in `dataset/` :

```
cp ../../lezione-08-semantic-search/scripts/dataset/corpus_* dataset/
```

(adattate il percorso a dove avete la cartella; se non lo trovate, la riserva già in `dataset/` è **identica**)

`TODO 1 — insieme`

`apri_collection()`

Client **persistente** su `scripts/chroma/` + collection `lumen` con metrica **cosine**. Poi nel main, sotto il commento del quick start (lo scheletro stampa già corpus e count): upsert dei **primi 3 doc** (embedding dall'npz: **espliciti!**) e una query di prova.

Fatto quando

Rilanciate lo script: i documenti **sono ancora lì**, e il count **non raddoppia** (upsert!). Niente npz da ricaricare: il DB è appena nato — guardate la cartella `chroma/` in `scripts/` .



⌨️ Lab 1 — il corpus nel DB

⌨️ **TOCCA A VOI — docente in affiancamento**

`working/vectorstore.py` · 75 minuti

si torna alle slide
per il debrief (poi ☕)

`TODO 2 · carica_corpus_nel_db(collection)`

Upsert dei **48 doc**: id, testo, embedding dall'npz, metadata `tipo`. L'ordine giusto sta in `corpus_meta.json` — come nella lezione scorsa.

`TODO 3 · search(query, k)`

I k più simili **chiesti al DB**, risultati con la faccia della lezione scorsa: id, tipo, testo, **score** (il DB vi dà *distanze*…).

Prescritto poco, apposta

Fissati i **nomi**, le variabili del main (le **stesse tre query della lezione scorsa** + `K`) e lo scopo: il **come è vostro**. Il main è già scritto a stadi: ogni TODO fatto accende il suo output (`[--]` = TODO che manca). **Finito prima?** Lo stretch è in `CONSEGNA.md`: variate `K`, query vostre per famiglia.

Fatto quando

`count()` dice **48** — e RESTA 48 al rilancio. E sulle tre query, la vostra `search()` e il vostro brute-force della lezione scorsa (`bruteforce.search`, già importato) mettono in testa **gli stessi documenti**, fianco a fianco.



📊 Al rientro — il confronto DB vs brute-force

Le tre query del main, sui due motori. Identici — score compresi:

Query	dal DB	brute-force Modulo 2 · Lezione 8
«Quanto costa la spedizione?»	`faq-02` **0.756**	`faq-02` **0.756**
«informazioni sulla scrivania»	`prod-01` **0.595**	`prod-01` **0.595**
«problemi con il motore…»	`tic-02` **0.588**	`tic-02` **0.588**

«E allora cosa abbiamo guadagnato?»

① la **velocità a scala** (il grafico che diverge) · ② la **persistenza** (il riavvio del quick start) · ③ e una cosa che **non avete ancora visto**: i FILTRI — subito dopo la pausa.

Se i due motori NON coincidono

È quasi sempre l'**embedder di default** che si è infilato: controllate che upsert e query viaggino coi **vostri** embedding (e col backend del meta). Se al primo lancio è partito un download di **~80 MB**, è lui.

☕ **Pausa — 15 minuti.** Al rientro: filtri e CRUD (Lab 2).



⌨ Lab 2 — filtri e CRUD

⌨ **TOCCA A VOI — docente in affiancamento**

`working/vectorstore.py · 60 minuti`

si torna alle slide
per il debrief

`TODO 4 · search_filtrata(query, k, tipo)`

La stessa ricerca, ma **solo tra i doc di un tipo**: il filtro è un parametro di `collection.query` — `where={"tipo": …}`. Provate «problemi con il motore della scrivania» **senza filtro** e poi con `"faq"` / `"ticket"` / `"recensione"`: la lista cambia faccia?

`TODO 5 · le prove CRUD, in coda al main`

delete di un doc che sta in top-3 (`collection.delete(ids=[…])`) → ri-query: sparito? Poi l'**update fatto bene**: nuovo testo per lo **stesso id**, ri-embed con `embed()`, upsert → su quale query esce adesso? Quale doc e come riscriverlo è affar vostro.

Fatto quando

Col filtro escono **solo** doc del tipo scelto · il doc cancellato **non esce più** · quello aggiornato **cambia query di appartenenza**.



📊 Al rientro — filtri e update

Il filtro cambia la domanda

«problemi con il motore» **senza filtro** è un CORO: un ticket arrabbiato (**0.588**), una recensione delusa (0.574), una FAQ rassicurante (0.557). Con `tipo="faq"` : **solo risposte ufficiali**.

«Cerco tra le risposte dell'azienda o tra le voci dei clienti?»

Il caso d'uso vero: il chatbot di supporto pesca **dalle FAQ**, non dai ticket degli altri clienti. Il recinto lo decidono i **metadata** caricati all'upsert.

L'update fatto bene

Il DB NON si accorge dei testi cambiati

Cambi il testo senza ri-embeddare → la ricerca resta sul significato **VECCHIO**. È il bug classico: pipeline che aggiorna i testi ma non i vettori, e restituisce risultati che non c'entrano più.

nuovo testo → ri-embed → upsert, stesso id

`faq-02` riscritta (ritiro in negozio) esce su «posso ritirare in negozio?» a **0.718** — e sparisce da «spedizione».



🎤 Demo — pgvector (si guarda)

🎤 **DEMO DEL DOCENTE — si guarda e si fanno domande**

`terminale psql del docente · 15 minuti`

si torna alle slide
per Pinecone e i criteri di scelta

Cosa state per vedere:

`① UNA TABELLA`

Il nostro corpus in **Postgres**: id, tipo, testo… e una colonna `vector(1536)`.

`② LA TOP-K IN SQL`

La ricerca del **Lab 1**, scritta come una SELECT.

`③ IL FILTRO = WHERE`

Il `where` del **Lab 2**, che qui è… una `WHERE`.

Postgres gira in un container **del docente**: voi oggi non installate nulla. Se un giorno vi serve, il giro completo è nel materiale della lezione.



ChromaDB e pgvector a confronto

Il parallelo, riga per riga — quello che avete appena visto: è il Postgres che conoscete, con un tipo in più.

ChromaDB (stamattina)	pgvector
collection	tabella
`upsert( ... )`	`INSERT` / `UPDATE`
`query(embedding, n_results=3)`	`ORDER BY embedding ⟺ q LIMIT 3`
`where={"tipo": "faq"}`	`WHERE tipo = 'faq'`
indice ANN (dietro le quinte)	`CREATE INDEX … USING hnsw`

Stessi risultati del Lab 1

`tic-02` 0.588 · `rec-04` 0.574 · `faq-06` 0.557 — **terzo motore, stessa ricerca**.

Il punto di forza vero

Quella WHERE può fare **JOIN coi TUOI dati** — ordini, utenti, permessi. Se hai già Postgres, i vettori stanno **dentro** il tuo database, transazioni comprese.

`⟺` è la distanza **cosine** di pgvector (ci sono anche `⟷` euclidea e `<#>` inner product): stessa storia della metrica scelta alla creazione.



Pinecone

La terza famiglia: il vector DB **gestito**, come servizio.

COME FUNZIONA

Crei un **index** dalla console web, mandi vettori e query **via API**: scalare è compito del servizio. Niente da installare, niente da amministrare.

I confini

Si **paga a consumo** · i dati stanno **da loro** (domanda da farsi coi dati sensibili) · sei **legato al servizio**.

Il modello dati è lo stesso

index ≈ collection, vettori, metadata, top-k, filtri: i concetti di stamattina valgono anche lì.



La tabella di scelta

Ora che le avete toccate tutte e tre — situazione → scelta:

La situazione	La scelta
Prototipo, corso, POC, dati locali	**ChromaDB** — embedded, zero infrastruttura
Hai già Postgres; i vettori devono stare coi TUOI dati (JOIN, transazioni, permessi)	**pgvector**
Scala gestita, zero operazioni, budget per pagarla	**Pinecone**
Carico serio, ma i dati restano tuoi	server dedicato: **Weaviate, Qdrant, Milvus**

La domanda giusta

Non «qual è il migliore» — ma: **chi lo amministra? dove stanno i dati? con cosa deve fare JOIN?**



L’ultimo blocco — lo sceglie il docente



ULTIMO BLOCCO — due strade, si decide adesso

25 minuti · in base a com'è andata la mattinata

si chiude insieme col wrap-up

STRADA A · 🗣 Q&A APERTO

Le domande che non sono uscite durante i lab — su oggi, ma anche su **tutto il percorso** (Modulo 2 · Lezione 1–8): API e costi, data engineering, embeddings e ricerca.

nessuna domanda è fuori tema

STRADA B · ⌨ IL 49° DOCUMENTO

Il corpus ha 48 voci: scrivetene **una nuova voi** — una FAQ che manca, un ticket realistico. Embed con `embed()`, upsert con **id nuovo** e il **tipo giusto**. Sempre in `working/vectorstore.py`.

Fatto quando: `count()` dice **49** · la vostra query la trova **in top-1** (provate con parole diverse dal testo) · col filtro del tipo giusto esce, con quello sbagliato no.

Riepilogo e ponte a Modulo 2 · Lezione 10

⚡ L'indice non scala con n

152.5 ms → 2.6 ms su 50k vettori. E si **paga alla scrittura**, una volta.

📦 La collection tiene tutto insieme

id + testo + embedding + metadata in un record: **basta file da allineare a mano**.

🔍 I filtri cambiano il recinto

Restringono l'insieme su cui si cerca: FAQ o voci dei clienti — lo decidono i **metadata**.

✏️ Update = ri-embed + upsert

Stesso id, embedding NUOVO: testo e vettore li sincronizzate **voi**.

Modulo 2 · Lezione 10 — arrivano i documenti LUNGHI

Finora 48 testi corti, **interi**. I PDF veri sono lunghi: non stanno in un unico embedding fatto bene, e retrievano male → vanno **spezzati**. Come si spezzano *bene* è la prossima lezione.

> Il `vectorstore.py` scritto oggi lo ritroverete nella prossima lezione dentro **aikit/**, riscritto in forma generica.



Grazie

Modulo 2 · Lezione 10: Chunking Strategies

